

Ethernet на плате JC1060P470C

Распиновка интерфейса RMII и чипа IP101: полный маппинг GPIO, инициализация драйвера в ESP-IDF v6, сравнение со SPI-модулями W5500 / DM9051 и диагностика bring-up.

Проект feature/ethernet-browser

Браузер на устройстве · JC1060P470C I_W_Y · экран 1024×600

Содержание

Глава 1. Обзор: как интернет устроен на плате	1
1.1. Доказательства из вендорских демо	1
1.2. Кто есть кто в сетевом тракте	1
Глава 2. Аппаратная основа: PHY IP101 и интерфейс RMI	2
2.1. Сигналы RMI	2
2.2. Тактирование: откуда берутся 50 МГц	3
2.3. Адрес PHY на шине MDIO	3
Глава 3. Распиновка платы: полный маппинг	4
3.1. Как пользоваться таблицей	5
Глава 4. Инициализация Ethernet в ESP-IDF v6	6
4.1. Подготовка: netif и цикл событий	6
4.2. Обработчики событий	7
4.3. Разбор стека: как кадр доходит до lwIP	8
Глава 5. Включение в проекте feature/ethernet-browser	10
5.1. Пошаговая миграция	10
5.2. Что станет с браузером	11
Глава 6. SPI-модули Ethernet: W5500, DM9051, KSZ8851SNL	11
6.1. Паттерн инициализации на примере W5500	11
Глава 7. Сравнение чипов и сценариев выбора	12
Глава 8. Диагностика и чек-лист bring-up	13
8.1. Чек-лист перед включением	13
8.2. Типовые симптомы	14
8.3. Успешный bring-up: как выглядит лог	14
Глава 9. Альтернатива: Wi-Fi через сопроцессор ESP32-C6	15
9.1. Минимальная инициализация STA	15
9.2. Что выбрать для браузера	15

Глава 1. Обзор: как интернет устроен на плате

Плата **JC1060P470C I_W_Y** (ESP32-P4 + дисплей 1024×600) получает доступ к сети двумя независимыми путями. Проводной путь построен на классической связке: **встроенный Ethernet-MAC (GMAC) внутри ESP32-P4** плюс внешний физический приёмопередатчик (PHY) **IP101**, соединённые интерфейсом **RMII**. Беспроводной путь обеспечивает **сопроцессор ESP32-C6**, с которым основной чип общается через компоненты **esp_hosted** и **esp_wifi_remote**. Оба пути сходятся в один сетевой стек **lwIP**, поэтому прикладной код (сокеты, HTTPS, DHCP) не зависит от того, какой из интерфейсов сейчас активен.

Это принципиально меняет картину по сравнению с тем, что мы предполагали раньше. На плате **нет ни W5500, ни DM9051, ни любого другого SPI-Ethernet контроллера** — это подтверждено схемой из репозитория производителя (5-Schematic/JC1060P470C_I_W_Y-V1.0.pdf), sdkconfig-файлами обоих вендорских демо и неудачным чтением регистра **VERSIONR** при попытке работы с W5500 на реальном железе. SPI-транзакции уходили «в пустоту», потому что на шине некому отвечать.

1.1. Доказательства из вендорских демо

В репозитории **megavatt05/ESP32P4-JC1060P470C-I_W_Y** лежат два проекта, использующих сеть: демо **esp_brookesia_phone** (телефонный UI с настройками) и отдельный пример **ethernet/basic**. Оба собраны с идентичной конфигурацией, которую видно прямо в закоммиченном **sdkconfig**:

```
CONFIG_EXAMPLE_USE_INTERNAL_ETHERNET=y    # встроенный EMAC, не SPI
CONFIG_EXAMPLE_ETH_PHY_IP101=y           # PHY-чип IP101
CONFIG_EXAMPLE_ETH_MDC_GPIO=31            # такт SMI
CONFIG_EXAMPLE_ETH_MDIO_GPIO=52           # данные SMI
CONFIG_EXAMPLE_ETH_PHY_RST_GPIO=51        # сброс PHY
CONFIG_EXAMPLE_ETH_PHY_ADDR=1             # адрес на шине MDIO
# CONFIG_EXAMPLE_USE_SPI_ETHERNET is not set # SPI-модули выключены
```

Строки **CONFIG_ETH_USE_ESP32_EMAC=y** и **CONFIG_ETH_PHY_INTERFACE_RMII=y** в тех же файлах фиксируют режим работы MAC. Кроме того, в демо **esp_brookesia_phone** приложение **Settings** содержит метод **initEthernet()**, вызывающий ровно этот стек инициализации по кнопке в интерфейсе, — то есть конфигурация не просто объявлена, а реально используется на этой плате.

1.2. Кто есть кто в сетевом тракте

Элемент платы	Чип / модуль	Роль
MAC-контроллер	GMAC внутри ESP32-P4	Формирование/разбор кадров Ethernet, DMA, интеграция с lwIP через esp_netif
PHY-трансивер	IP101G/LF (IC+, 10/100)	Кодирование сигнала, автосогласование 10/100 Мбит, полный дуплекс
Разъём	RJ45 HR6213	Витая пара, магнитная развязка и индикаторные светодиоды встроены в разъём
Тактирование PHY	Кварц 25 МГц X1	Из него PHY синтезирует 50 МГц REF_CLK для RMI
Wi-Fi сопроцессор	ESP32-C6	Беспроводная сеть через esp_hosted; проводного отношения к Ethernet не имеет

Таблица 1. Состав сетевого тракта платы JC1060P470C

W5500 на этой плате отсутствует

Попытка инициализации W5500 по SPI дала VERSIONR = 0x00 — шина отвечала тишиной, потому что чипа физически нет. Все SPI-пины (CS=10, SCLK=12, MOSI=11, MISO=13) были догадкой и к реальной плате отношения не имеют. Правильный путь — внутренний EMAC + IP101, описанный в этой документации.

Глава 2. Аппаратная основа: PHY IP101 и интерфейс RMI

В Ethernet-приемнике на кристалле Espressif функции разделены строго: **MAC** (Media Access Control) управляет кадрами, адресами и очередями DMA, а **PHY** (Physical Layer) занимается физикой — кодированием битов в электрические уровни витой пары. В ESP32-P4 MAC встроен в кристалл (GMAC), а PHY вынесен наружу — на плате это IP101G/LF производства IC+. Связывает их интерфейс **RMI** (Reduced Media Independent Interface) — упрощённый вариант MII с общим тактовым сигналом 50 МГц и укороченными шинами данных.

2.1. Сигналы RMI

Сигнал	Направление	Назначение
TXD0, TXD1	P4 → PHY	Два бита данных на передачу за такт REF_CLK
TX_EN	P4 → PHY	Активен, когда на TXD идут валидные биты кадра
RXD0, RXD1	PHY → P4	Два бита данных на приём
CRS_DV	PHY → P4	Несущая обнаружена + данные валидны (объединённый сигнал)

Сигнал	Направление	Назначение
REF_CLK	PHY → P4	Такт 50 МГц, общий для обеих сторон; в этой схеме генерирует PHY
MDC	P4 → PHY	Такт регистрационного интерфейса SMI (управление, до ~2.5 МГц)
MDIO	двунапр.	Данные SMI: чтение/запись регистров PHY (статус линка, дуплекс, скорость)

Таблица 2. Сигналы интерфейса RMI между ESP32-P4 и IP101

Важно различать **шины данных** (TXD/RXD/CRS_DV/REF_CLK — они тактируются 50 МГц и передают кадры) и **управляющую шину SMI** (MDC/MDIO — через неё драйвер читает идентификатор PHY, статус линка и управляет автосогласованием). Шина данных работает постоянно в реальном времени, а SMI обращается к PHY эпизодически — именно поэтому MDC/MDIO можно развести на любые удобные GPIO через матрицу, тогда как линии данных привязаны к фиксированным pads GMAC.

2.2. Тактирование: откуда берутся 50 МГц

На плате источник такта — сам PHY. Кварц 25 МГц (X1, выводы XI/XO — пины 1 и 2 IP101) питает внутренний генератор, из которого PLL формирует 50 МГц; готовая частота выходит на выводе **50M_CLKO** (пин 14) и заводится в ESP32-P4 на GPIO50 — это pad **GMAC_RMI_CLK** (функция F3 по таблице 2-3 даташита ESP32-P4). Схема «PHY — мастер тактов» самая надёжная: одна точка генерации, нулевой фазовый сдвиг между данными и тактом, и MAC-у не нужно ничего конфигурировать — режим по умолчанию `ETH_ESP32_ETHMAC_DEFAULT_CONFIG()` принимает внешний REF_CLK.

Для справки: у ESP32-P4 есть и обратный вариант — сам P4 умеет выдавать 50 МГц наружу через pads REF_50M_CLK (GPIO23/GPIO39) на вход 50M_CLKI PHY. На данной плате он не используется, но полезно знать при чтении чужих схемотехник: если кварца на PHY нет, значит тактирует P4.

2.3. Адрес PHY на шине MDIO

На шине SMI может висеть до 32 PHY, каждый со своим адресом 0-31. У IP101 адрес зашивается подтяжками выводов **PHY_AD0...PHY_AD3** во время сброса. На плате вывод PHY_AD0 (пин 11) подтянут резистором 5.1 кОм к питанию PHY_3V3, остальные — к земле, итоговый адрес = **1**. Именно это значение стоит в `CONFIG_EXAMPLE_ETH_PHY_ADDR=1` вендорских демо и в наших Kconfig-настройках. Если при bring-up драйвер не находит PHY или читает мусорный ID — первым делом сверяют адрес.

Глава 3. Распиновка платы: полный маппинг

Ниже — полный маппинг Ethernet-тракта, собранный из схемы JC1060P470C_I_W_Y-V1.0.pdf (узел PHY U4 и разъём JP1 модуля JC-ESP32P4-M3) и сверенный с таблицей 2-3 даташита ESP32-P4 (колонка функций F3). Линии данных RMI сидят на фиксированных pads GMAC, а MDC/MDIO разведены на GPIO31/52 через GPIO-матрицу — это стандартное решение Espressif, совпадающее с умолчаниями их примеров для P4.

Сигнал	GPIO ESP32-P4	Пин IP101	Примечание
TXD0	34	9	Данные на передачу, бит 0 (pad GMAC_PHY_TXD0)
TXD1	35	8	Данные на передачу, бит 1 (pad GMAC_PHY_TXD1)
TX_EN	49	5	Разрешение передачи (pad GMAC_PHY_TXEN)
RXD0	29	18	Данные на приём, бит 0 (pad GMAC_PHY_RXD0)
RXD1	30	17	Данные на приём, бит 1 (pad GMAC_PHY_RXD1)
CRS_DV	28	19	Валидность приёма (pad GMAC_PHY_RXDV)
REF_CLK	50	14	50 МГц от выхода 50M_CLKO PHY; вход GMAC_RMII_CLK
MDC	31	22	Такт SMI — через GPIO-матрицу (умолчание примеров IDF)
MDIO	52	23	Данные SMI — через GPIO-матрицу (умолчание примеров IDF)
PHY_RSTN	51	32	Активный низкий сброс PHY; управляет драйвер esp_eth

Таблица 3. Маппинг Ethernet на плате JC1060P470C (PHY addr = 1)

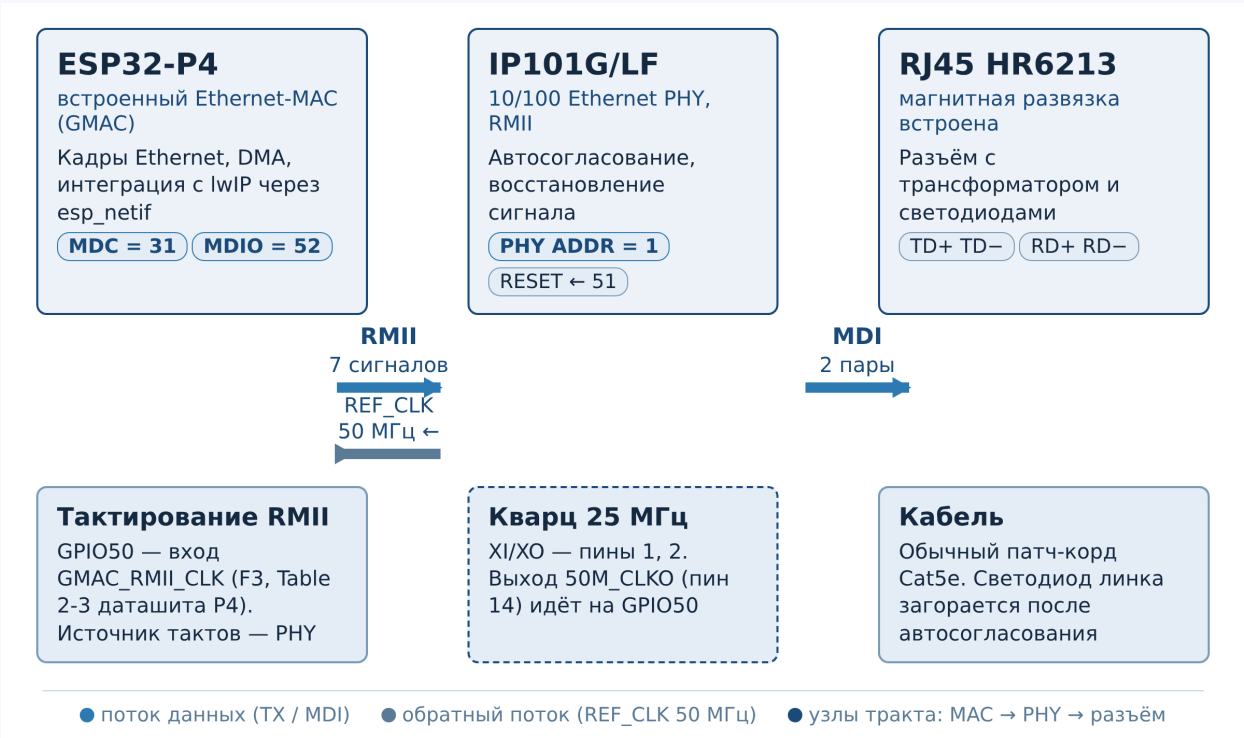


Рисунок 1. Тракт данных: MAC ESP32-P4 — PHY IP101 — разъём RJ45

3.1. Как пользоваться таблицей

При диагностике или доработке платы таблица отвечает на три практических вопроса. **Осциллографом:** проверять наличие 50 МГц на GPIO50 и активность TX_EN на GPIO49 во время передачи. **Логически:** помнить, что GPIO31/52 заняты SMI и их нельзя переиспользовать под другие функции (GPIO31 имеет также роль SPI2_Q — конфликт легко получить случайно при добавлении второй SPI-периферии). **При разводке плат-расширений:** линии REF_CLK — высокочастотные, их длина и земля должны идти коротко и без прорезей.

Полная схема доступна в репозитории производителя: папка **5-Schematic**, файл **JC1060P470C_I_W_Y-V1.0.pdf**. Узел PHY подписан как U4, разъём RJ45 — RJ1, межмодульный разъём — JP1. Все номиналы подтяжек (5.1 кОм на PHY_AD0, 6.19 кОм ISET, 75 Ом на MDI) видны на том же листе.

Глава 4. Инициализация Ethernet в ESP-IDF v6

В ESP-IDF v6 связка «встроенный MAC + обычный PHY» осталась в базовом компоненте `esp_eth` — в отличие от SPI-контроллеров (W5500, DM9051, KSZ8851SNL), которые вынесены в managed-компоненты. Значит для IP101 ничего скачивать не нужно: достаточно `CONFIG_ETH_USE_ESP32_ETHMAC=y`, который на P4 включён по умолчанию, когда SoC поддерживает EMAC. API в v6.0.3 совпадает с v5.3/v5.4, на котором собраны вендорские демо — паттерн ниже взят из их компонента `ethernet_init` и проверен производителем платы.

4.1. Подготовка: netif и цикл событий

Перед инициализацией драйвера нужно поднять сетевой слой: инициализировать `esp_netif` и создать дефолтный цикл событий. В нашем проекте это делает `app_main()` до вызова сетевого стартера. Затем выполняется сама последовательность из шести шагов — она полностью линейная и обрабатывается ESP_ERROR_CHECK-ами: любая ошибка на этом этапе фатальна и означает проблему в конфигурации, а не в сети.

```
#include "esp_eth.h"
#include "esp_netif.h"
#include "esp_event.h"

static esp_eth_handle_t s_eth_handle = NULL;
static esp_netif_t      *s_eth_netif = NULL;

esp_err_t app_eth_init(void)
{
    // 0. В app_main() до этого места:
    //   ESP_ERROR_CHECK(esp_netif_init());
    //   ESP_ERROR_CHECK(esp_event_loop_create_default());

    // 1. Конфиг MAC + PHY: значения с платы JC1060P470C
    eth_mac_config_t mac_config = ETH_MAC_DEFAULT_CONFIG();
    eth_phy_config_t phy_config = ETH_PHY_DEFAULT_CONFIG();
    phy_config.phy_addr = 1;           // PHY_AD0 подтянут к 3V3 (см. главу 2)
    phy_config.reset_gpio_num = 51;    // PHY_RSTN

    eth_esp32_emac_config_t emac_config = ETH_ESP32_ETHMAC_DEFAULT_CONFIG();
    emac_config.smi_gpio.mdc_num = 31; // MDC
    emac_config.smi_gpio.mdio_num = 52; // MDIO

    // 2. Объекты MAC (внутренний GMAC) и PHY (IP101)
    esp_eth_mac_t *mac = esp_eth_mac_new_esp32(&emac_config, &mac_config);
    esp_eth_phy_t *phy = esp_eth_phy_new_ip101(&phy_config);

    // 3. Установка драйвера
    esp_eth_config_t eth_config = ETH_DEFAULT_CONFIG(mac, phy);
    ESP_ERROR_CHECK(esp_eth_driver_install(&eth_config, &s_eth_handle));

    // 4. Сетевой интерфейс поверх lwIP
    esp_netif_config_t netif_cfg = ESP_NETIF_DEFAULT_ETH();
    s_eth_netif = esp_netif_new(&netif_cfg);
```

```

// 5. Мост "драйвер <-> стек": glue пересылает события и кадры
esp_eth_netif_glue_handle_t glue = esp_eth_new_netif_glue(s_eth_handle);
ESP_ERROR_CHECK(esp_netif_attach(s_eth_netif, glue));

// 6. Пользовательские обработчики (лог, флаг "сеть готова")
ESP_ERROR_CHECK(esp_event_handler_register(ETH_EVENT, ESP_EVENT_ANY_ID,
    &event_handler, NULL));
ESP_ERROR_CHECK(esp_event_handler_register(IP_EVENT, IP_EVENT_ETH_GOT_IP,
    &got_ip_event_handler, NULL));

// 7. Старт: драйвер снимет reset с PHY, начнётся автосогласование
ESP_ERROR_CHECK(esp_eth_start(s_eth_handle));
return ESP_OK;
}

```

4.2. Обработчики событий

Драйвер и стек общаются с прикладом исключительно через события. Их четыре уровня важности: физический линк (ETHERNET_EVENT_CONNECTED/DISCONNECTED), жизненный цикл драйвера (START/STOP) и главное событие приложения — IP_EVENT_ETH_GOT_IP, после которого разрешены любые TCP/UDP-операции. В браузере это событие снимает блокировку с web-клиента; до него UI работает в автономном режиме.

```

static const char *TAG = "app_eth";

static void eth_event_handler(void *arg, esp_event_base_t base,
    int32_t id, void *data)
{
    switch (id) {
        case ETHERNET_EVENT_CONNECTED:
            ESP_LOGI(TAG, "Link Up: 10/100, полный дуплекс");
            break;
        case ETHERNET_EVENT_DISCONNECTED:
            ESP_LOGW(TAG, "Link Down: кабель вынут");
            break;
        case ETHERNET_EVENT_START:
            ESP_LOGI(TAG, "Драйвер запущен");
            break;
        case ETHERNET_EVENT_STOP:
            ESP_LOGI(TAG, "Драйвер остановлен");
            break;
        default:
            break;
    }
}

static void got_ip_event_handler(void *arg, esp_event_base_t base,
    int32_t id, void *data)
{
    ip_event_got_ip_t *event = (ip_event_got_ip_t *)data;
    const esp_netif_ip_info_t *ip = &event->ip_info;
    ESP_LOGI(TAG, "Got IP: " IPSTR " mask " IPSTR " gw " IPSTR,
        IP2STR(&ip->ip), IP2STR(&ip->netmask), IP2STR(&ip->gw));
}

```

```
// здесь браузеру можно разрешать HTTPS-запросы  
}
```

4.3. Разбор стека: как кадр доходит до lwIP

Инициализация создаёт три слоя. Нижний — **драйвер**: пара объектов MAC и PHY, скрытая за единым `esp_eth_handle_t`; MAC знает рег-ы GMAC и DMA, PHY — регистры IP101 на шине SMI. Средний — **netif glue**: компонент-адаптер, который подписывается на внутренние события драйвера, транслирует их в действия `esp_netif` (например, старт DHCP-клиента при подключении) и прокидывает исходящие/входящие кадры между DMA и lwIP. Верхний — **esp_netif**: абстракция сетевого интерфейса, единая для Ethernet и Wi-Fi; у неё lwIP берёт адрес, маршруты и сокеты.

Фаза 1 — установка драйвера

net/app_eth.c

1. Настроить конфиг MAC

```
ETH_ESP32_ETH_DEFAULT_CONFIG()
smi_gpio.mdc_num = 31, mdio_num = 52
```

**2. Настроить конфиг PHY**

```
phy_config.phy_addr = 1; reset_gpio_num = 51
```

**3. Создать объекты MAC и PHY**

```
esp_eth_mac_new_esp32(&emac_cfg, &mac_cfg)
esp_eth_phy_new_ip101(&phy_cfg)
```

**4. Установить драйвер**

```
esp_eth_config_t cfg = ETH_DEFAULT_CONFIG(mac, phy)
esp_eth_driver_install(&cfg, &handle)
```

**5. Создать сетевой интерфейс**

```
esp_netif_new(ESP_NETIF_DEFAULT_ETH()) — экземпляр esp_netif
поверх lwIP
```

**6. Привязать драйвер к интерфейсу**

```
esp_eth_new_netif_glue(handle); esp_netif_attach(netif, glue)
```

**7. Подписаться на события**

```
ETH_EVENT (ANY_ID) и IP_EVENT / IP_EVENT_ETH_GOT_IP
```

Фаза 2 — запуск и получение адреса**8. Запустить Ethernet**

```
esp_eth_start(handle) — PHY выходит из reset, начинается
автосогласование
```

**9. Дождаться линка**

```
Событие ETHERNET_EVENT_CONNECTED (10/100 Мбит, полный дуплекс)
```

**10. DHCP: запрос адреса**

```
lwIP стартует DHCP-клиент: DISCOVER → OFFER → REQUEST → ACK
```

**11. Событие IP_EVENT_ETH_GOT_IP**

```
Обработчик получает ip_info.ip — можно открывать соединения
(для браузера: HTTPS-запросы)
```

Рисунок 2. Последовательность инициализации и цепочка событий до получения IP

После `esp_eth_start()` события идут цепочкой: драйвер снимает `reset` (GPIO51), PHY поднимает такт и начинает автосогласование; получив линк, PHY сообщает о нём через MDIO, и драйвер публикует `ETHERNET_EVENT_CONNECTED`. На это событие `glue` вызывает внутри `esp_netif_action_connected()`, тот стартует DHCP-клиент, и когда сервер выдаст аренду, `esp_netif` публикует `IP_EVENT_ETH_GOT_IP`. Вся цепочка занимает обычно 1-3 секунды после вставки кабеля.

DHCP по умолчанию, статика — если нужна

DHCP стартует автоматически и отдельного кода не требует. Статический адрес задаётся явно: `esp_netif_dhcpc_stop(netif)`, затем `esp_netif_set_ip_info(netif, &ip_info)` с заполненной структурой, затем при необходимости `esp_netif_dhcpc_start()`. Для браузера на устройстве достаточно DHCP.

Глава 5. Включение в проекте feature/ethernet-browser

В нашей ветке сетевой слой изначально был написан под W5500: компонент `espressif/w5500` ^2.0.0 в `idf_component.yml`, SPI-пины в `Kconfig` (`EB_W5500_*`), установка GPIO ISR-сервиса для линии прерываний. На плате JC1060P470C всё это не нужно — глава 1 объяснила почему. Ниже план перехода на встроенный EMAC, согласованный с распиновкой главы 3.

5.1. Пошаговая миграция

1. В `main/idf_component.yml` удалить зависимость `espressif/w5500` (и не добавлять ничего взамен — `esp_eth` уже в составе IDF).
2. В `main/Kconfig.projbuild` заменить группу `EB_W5500_*` на четыре параметра:
`EB_ETH_MDC=31`, `EB_ETH_MDIO=52`, `EB_ETH_PHY_RST=51`, `EB_ETH_PHY_ADDR=1`.
 Линия INT не нужна вовсе.
3. В `main/net/app_eth.c` переписать инициализацию по образцу главы 4: `esp_eth_mac_new_esp32` + `esp_eth_phy_new_ip101`; удалить `spi_bus_initialize`, `gpio_install_isr_service` и обработчик прерывания.
4. В `sdkconfig` проверить наличие `CONFIG_ETH_USE_ESP32_EMAC=y` (после `fullclean` и `menuconfig` он подтянется автоматически, т.к. `SOC_EMAC_SUPPORTED` для P4).
5. Выполнить **idf.py fullconf** и **пересборку**: `idf.py fullclean && idf.py build` — старый managed-компонент `w5500` и его символы уйдут.
6. На устройстве: вставить патч-корд в RJ45 до подачи питания либо после — драйвер отслеживает линк динамически.

5.2. Что станет с браузером

Браузерный поток остаётся без изменений логически: он и так построен вокруг флага «сеть готова». Меняется источник флага: вместо poll-статуса W5500 браузер ждёт IP_EVENT_ETH_GOT_IP (обработчик из главы 4 ставит event-group бит NET_READY). До события UI показывает сплэш «нет сети», после — разрешает HTTPS-запросы web_client. Таймаут ожидания 15-20 секунд с переходом в view-only режим сохраняем: если пользователь включил устройство без кабеля, прошивка не должна зависать на ожидании.

Порядок старта в app_main

app_lcd_init → подсветка → browser_start (UI рисуется сразу) → app_eth_init в фоне. Не блокируйте app_main ожиданием IP: экран должен ожить мгновенно, а сеть догрузится параллельно и «включит» адресную строку.

Глава 6. SPI-модули Ethernet: W5500, DM9051, KSZ8851SNL

SPI-контроллеры — это чипы, совмещающие MAC и PHY, подключаемые по обычной SPI-шине. Их выбирают, когда в SoC нет своего EMAC (ESP32-C3/C6/C2), когда линии MAC уже заняты, когда нужен второй независимый интерфейс, или когда плату разводят под жёсткую гальваническую изоляцию (модуль с трансформатором и корпусом). В ESP-IDF v6 их драйверы вынесены в managed-компоненты: espressif/w5500, espressif/dm9051, espressif/ksz8851: они подключаются через idf_component.yml и по API повторяют друг друга.

6.1. Паттерн инициализации на примере W5500

```
// Зависимость: espressif/w5500 ^2.0.0 (для IDF >= 6.0)
#include "esp_eth.h"
#include "driver/spi_master.h"
#include "driver/gpio.h"

esp_err_t app_eth_w5500_init(void)
{
    // 1. Прерывание: глобальный ISR-сервис обязателен до gpio_isr_handler_add
    ESP_ERROR_CHECK(gpio_install_isr_service(0)); // ESP_ERR_INVALID_STATE = уже стоит

    // 2. SPI-шина
    spi_bus_config_t buscfg = {
        .miso_io_num = 13, .mosi_io_num = 11, .sclk_io_num = 12,
        .quadwp_io_num = -1, .quadhd_io_num = -1,
    };
    ESP_ERROR_CHECK(spi_bus_initialize(SPI2_HOST, &buscfg, SPI_DMA_CH_AUTO));

    // 3. Конфиг устройства на шине
    spi_device_interface_config_t devcfg = {
```

```

        .mode = 0,
        .clock_speed_hz = 16 * 1000 * 1000,    // 16 МГц, можно до 33-40
        .queue_size = 20,
        .spics_io_num = 10,                      // CS
    };

    // 4. Конфиг W5500: пин прерывания и период опроса (если INT = -1)
    eth_w5500_config_t w5500_config = ETH_W5500_DEFAULT_CONFIG(SPI2_HOST, &devcfg);
    w5500_config.int_gpio_num = 9;                // -1 -> режим polling
    w5500_config.poll_period_ms = 0;             // 0 при активном прерывании

    eth_mac_config_t mac_config = ETH_MAC_DEFAULT_CONFIG();
    eth_phy_config_t phy_config = ETH_PHY_DEFAULT_CONFIG();
    phy_config.reset_gpio_num = 14;              // RST модуля, -1 если нет
    phy_config.phy_addr = 1;

    esp_eth_mac_t *mac = esp_eth_mac_new_w5500(&w5500_config, &mac_config);
    esp_eth_phy_t *phy = esp_eth_phy_new_w5500(&phy_config);

    // 5. W5500 может не иметь заводского MAC — задаём локально-администрируемый
    uint8_t base_mac[6], local_mac[6];
    esp_efuse_mac_get_default(base_mac);
    esp_derive_local_mac(local_mac, base_mac);
    // ... после esp_eth_driver_install():
    // esp_eth_ioctl(eth_handle, ETH_CMD_S_MAC_ADDR, local_mac);
    return ESP_OK;
}

```

Здоровый W5500 отвечает на чтение регистра **VERSIONR** (0x0039) значением **0x04**; драйвер проверяет это при установке и отклоняет чип с другим ответом. DM9051 и KSZ8851SNL инициализируются идентично — меняются только префиксы функций (`esp_eth_mac_new_dm9051`, `esp_eth_mac_new_ksz8851snl`) и поля их конфигов. Дальнейшие шаги (`netif`, `glue`, события, DHCP) абсолютно те же, что в главе 4: верхняя часть стека не зависит от типа MAC.

Глава 7. Сравнение чипов и сценариев выбора

Чип	Интерфейс	Скорость	GPIO	Драйвер в IDF v6	Когда выбирать
GMAC + IP101 (эта плата)	RMII внутренний	100 Мбит	7 данных + 2 SMI + 1 RST	esp_eth (встроено)	Платы с EMAC: P4, классический ESP32; нулевой BOM
W5500	SPI	100 Мбит	4 (SPI) + INT + RST	espressif/w5500	SoC без EMAC (C3/C6), нужна простая разводка, модульные решения
DM9051	SPI	100 Мбит	4 (SPI) + INT + RST	espressif/dm9051	Аналог W5500; чаще встречается в промышленных модулях
KSZ8851S NL	SPI	100 Мбит	4 (SPI) + INT + RST	espressif/ksz8851	Расширенный температурный диапазон, автодокументированные промышленные проекты

Таблица 4. Сравнение вариантов Ethernet на ESP32

Для JC1060P470C выбор предопределён схемой: только встроенный путь даёт работу без дополнительных компонентов. Экономия ощутимая: RMII-вариант не занимает SPI-хост, не требует ISR-сервиса и не добавляет точку отказа на шине; кадры ходят через DMA напрямую в память, что даёт лучшую латентность и почти полную загрузку канала 100 Мбит. SPI-вариант выигрывает только там, где RMII физически недоступен, — то есть на других членах семейства ESP32, либо когда нужны два независимых интерфейса (EMAC + SPI-модуль одновременно — конфигурация, которую поддерживает и пример IDF, и драйверы).

Практическое следствие для отладки: если плата с W5500 «не видит чип» (VERSIONR = 0x00 или 0xFF), почти всегда виновата проводка: питание 3.3В, общая земля, переставленные MISO/MOSI, зависший CS. Но на JC1060P470C искать W5500 не нужно — его нет, и это не дефект, а штатная схемотехника.

Глава 8. Диагностика и чек-лист bring-up

8.1. Чек-лист перед включением

1. Питание PHY: на IP101 подано 3.3В (линия PHY_3V3), потребление ~50-80 мА.
2. Кабель: патч-корд Cat5e вставлен в RJ1, второй конец — в роутер/коммутатор с DHCP-сервером.
3. Kconfig: EB_ETH_MDC=31, EB_ETH_MDIO=52, EB_ETH_PHY_RST=51, EB_ETH_PHY_ADDR=1.
4. sdkconfig: CONFIG_ETH_USE_ESP32_EMAC=y (видно в menuconfig: Component config / Ethernet).
5. GPIO 28-31, 34, 35, 49-52 не заняты другими драйверами (дисплей, тач, SD — проверены).

6. Собрано после `idf.py fullclean` — символы `w5500` не остались в `sdkconfig`.
7. Логи на уровне `INFO`: видно старт драйвера и события линка.
8. Осциллограф (если есть): 50 МГц на `GPIO50` сразу после сброса `PHY`.

8.2. Типовые симптомы

Симптом	Вероятная причина	Решение
<code>esp_eth_driver_install:</code> <code>ESP_ERR_NO_MEM</code> или <code>assert</code>	Не включён <code>CONFIG_ETH_USE_ESP32_ETH</code>	<code>menuconfig</code> : Component config → Ethernet → Support ESP32 internal EMAC
В логе "PHY ID read failed" / ID = 0xFFFF	Неверный <code>phy_addr</code> или мёртвая шина MDIO	Сверить <code>addr=1</code> (подтяжка <code>PHY_AD0</code>), прозвонить <code>MDC=31/MDIO=52</code>
PHY ID читается, линка нет	Кабель/коммутатор, или PHY в <code>reset</code>	Проверить <code>GPIO51</code> (<code>RSTN</code> активен низко), кабель, светодиоды RJ45
Линк есть, IP не приходит	DHCP-сервер не выдаёт адрес	Проверить пул DHCP на роутере; задать статику вручную (см. главу 4)
IP есть, TCP-соединения рвутся	<code>REF_CLK</code> нестабилен (джиттер/отсутствие)	Осциллограф на <code>GPIO50</code> : ровные 50 МГц; проверить пайку X1
Пропуски пакетов под нагрузкой	Не хватает DMA-буферов	Увеличить <code>CONFIG_ETH_DMA_RX_BUFFER_NUM</code> (например, 20)
Обращение к <code>W5500</code> даёт <code>VERSIONR=0x00</code>	Чипа нет на шине (наша плата!) — либо проводка SPI	Перейти на EMAC+IP101 по главам 3-5; на других платах проверить MISO/MOSI/CS/питание
<code>GPIO31</code> конфликтует с <code>SPI2</code>	Вторая периферия захватила <code>rad</code>	Развести других потребителей с <code>GPIO31</code> (<code>SPI2_Q</code>); SMI приоритетен
Событий <code>CONNECTED</code> нет, но PHY живой	Драйвер ожидает прерывание PHY	Убедиться, что <code>reset_gpio_num=51</code> передан в <code>phy_config</code> (иначе PHY спит)

Таблица 5. Диагностика: симптом — причина — решение

8.3. Успешный bring-up: как выглядит лог

```
I (3120) app_eth: Драйвер запущен
I (3125) esp_eth.mac-esp: ... Ethernet Started
I (4800) app_eth: Link Up: 10/100, полный дуплекс
I (6310) app_eth: Got IP: 192.168.1.87 mask 255.255.255.0 gw 192.168.1.1
I (6315) browser: сеть готова, адресная строка разблокирована
```

Разброс времени между `Link Up` и `Got IP` — обычно 0.5-1.5 секунды (обмен DHCP). Если `Link Up` появляется мгновенно после вставки кабеля, а IP задерживается на десятки секунд — ищите

проблемы на DHCP-сервере, не в ESP. Полное отсутствие линка при живом PHY почти всегда означает физический уровень: кабель, магниты RJ45, или оторванную подтяжку RESET.

Глава 9. Альтернатива: Wi-Fi через сопроцессор ESP32-C6

На плате распаян модуль ESP32-C6, соединённый с P4 по UART (линии C6_U0TXD/C6_U0RXD и управление питанием C6_CHIP_PU видны на той же схеме). Espressif решает такую связку пакетом **esp_hosted**: на P4 работает виртуальный Wi-Fi-драйвер (`esp_wifi_remote`), который транслирует вызовы `esp_wifi` через UART/SDIO в реальный драйвер на C6. С точки зрения приложения это обычный Wi-Fi STA: те же события `WIFI_EVENT`, тот же `IP_EVENT_STA_GOT_IP`, тот же lwIP. Именно так устроено вендорское демо `esp_brookesia_phone` — его приложение Settings сканирует и подключается к Wi-Fi, а Ethernet включается отдельным тумблером.

9.1. Минимальная инициализация STA

```
// idf_component.yml: espressif/esp_wifi_remote, espressif/esp_hosted
ESP_ERROR_CHECK(esp_netif_init());
ESP_ERROR_CHECK(esp_event_loop_create_default());
esp_netif_create_default_wifi_sta();

wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
ESP_ERROR_CHECK(esp_wifi_init(&cfg));
ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_STA));
ESP_ERROR_CHECK(esp_wifi_start());
// далее: esp_wifi_scan_start() -> esp_wifi_set_config(ESP_IF_WIFI_STA, ...)
//         с SSID/паролем -> esp_wifi_connect()
// IP придёт в IP_EVENT_STA_GOT_IP — тот же паттерн, что и у Ethernet
```

9.2. Что выбрать для браузера

Критерий	Ethernet (EMAC + IP101)	Wi-Fi (ESP32-C6, esp_hosted)
Настройка	Нулевая: кабель в розетку	SSID и пароль вводить с экранной клавиатуры
Задержка и джиттер	Миллисекунды, стабильно	Зависит от радиоканала и загрузки эфира
Пропускная способность	Стабильные ~90+ Мбит	Десятки Мбит в реальных условиях
Зависимости	Только esp_eth (в IDF)	Два managed-компонента + транспорт UART
Мобильность	Привязан к кабелю	Свободное перемещение
Отказоустойчивость линки	Событие DISCONNECTED, авто-reconnect драйвера	Та же модель событий

Таблица 6. Сравнение сетевых путей на этой плате

Рекомендация для проекта: **основным делаем Ethernet** — он не требует настройки, быстрее и стабильнее для HTTPS-трафика браузера. Wi-Fi оставляем как второй путь: его уже умеет вендорский UI, а при желании в браузере можно добавить экран выбора сети. Прикладной код от типа интерфейса не зависит: оба пути публикуют одинаковые IP-события, и web_client работает поверх любого активного netif. Такой двойной путь — удобная страховка на время bring-up: если кабель ещё не подведён, устройство демонстрирует сеть по радиоканалу.